

RAG-Based Computer Science Chatbot

Vipul Dhotre, Ziya Qureshi, Ananya Sristy

April 19, 2026

1 Introduction

Through recent years, many companies have adopted the use of Artificial Intelligence to help their users navigate and streamline processes. Chatbot systems have become an integral part of many applications by automating support for clients. In colleges, new students frequently seek basic information, such as course and admission requirements, but must meet with academic advisors to understand those requirements. At times, wait times for these advisors can last hours which is what our project aims to resolve.

This project presents the development of a Retrieval-Augmented Generated (RAG) based Chatbot designed specifically for the Computer Science Department at Georgia State University (GSU). The primary objectives for this Chatbot are to provide accurate and source-based responses to old and incoming students using official GSU documents.

The RAG based Computer Science Chatbot uses document retrieval technique along with a locally hosted language model to ensure the responses are factual. In addition to this, the project works through performance optimizations and hybrid retrieval strategies to improve the response time as well as the accuracy. This report outlines the design, implementation and future work of this Chatbot system.

2 System Architecture

The Chatbot follows a Retrieval-Augmented Generation (RAG) architecture, which combines document retrieval with language model based response generation. The system pipeline consists of document processing, query processing, document retrieval, and response generation.

When documents are first added to the Chatbots sources, the document goes through pre-processing which consists of cleaning of the text in those documents to better assist the chunking process. The text is then split into chunks and placed in a vector-store.

When a user submits a query, the system first retrieves relevant documents using a hybrid retrieval strategy that combines dense (embedding-based) and sparse (keyword-based) methods. Dense retrieval is performed using vector similarity while sparse retrieval is implemented using the BM25 algorithm.

The retrieved documents are then passed to a Llama-based language model, Ollama 3.2, which generates responses based on the provided context of the documents. The system

ensures that responses are concise, based solely on the source documents, and include the source citation.

To improve system efficiency and performance, the vectorization process is separated into its own file, requiring only one run to initialize. This was an important step in our development process as it significantly reduced the processing time of our Chatbot.

3 Methodology

3.1 Data Collection

The dataset for this project consists of publicly available documents from the GSU website, including program pages, course catalogs, and departmental documents. These documents are loaded using a web loader to ensuring that the system uses the most relevant information as its context at runtime.

3.2 Text Processing

The collected documents were cleaned using a pre-processing function to remove unnecessary whitespace and streamline the formatting. The cleaned text was then divided into chunks using a text splitter. Text processing was an important process in our Chatbots development as it allows the creation of smaller information rather than processing whole documents.

3.3 Vectorization

Each document chunk was converted into a dense vector representation using an embedding model. These embeddings capture the meaning of the text and enable similarity-based retrieval.

To optimize performance, the vectorization process was implemented as a separate pre-processing step. The embeddings are stored locally using a FAISS vector database, eliminating the need to recompute embeddings during runtime. This optimization reduced response time from approximately 4–5 minutes per query to less than 30 seconds.

3.4 Hybrid Retrieval

The system uses a hybrid retrieval strategy that combines dense and sparse retrieval techniques. Dense retrieval is performed using FAISS with Maximum Marginal Relevance (MMR) and Sparse retrieval, implemented using the BM25 algorithm to capture keyword based matches. The output of both is combined and then de-duplicated to form the context for the LLM. A ranking step is done to increase the importance of certain documents.

3.5 Response Generation

The response generation of our Chatbot is done using a Llama based language model, specifically Ollama 3.2. A prompt template is used to restrict and guide the model through its

response generation such as avoiding explicit context. The model is also guardrailed to prevent answering questions out of context and respond with "I don't know" when an out of scope query is entered.

4 Implementation

The Chatbot system was implemented in Python using the LangChain framework. The system is modular, with separate components for pre-processing and query handling.

The pre-processing section is responsible for loading documents, cleaning text, splitting content into chunks, generating embeddings, and storing them in a FAISS vector database. The processed documents are also saved in JSON format for use in sparse retrieval. The main application module handles user queries, performs hybrid retrieval, re-ranks results, and generates responses using the language model.

5 Evaluation and Results

The system was evaluated based on response time and answer quality. Initially, the Chatbot experienced significant delays in response time due to vectorization being completed at each runtime. With the previous structure, the responses took around 5 to 6 minutes to compute.

After separating the vectorization process into a pre-processing step, response time improved significantly to approximately one minute per query. This optimization greatly enhanced system performance.

The introduction of hybrid retrieval further improved answer accuracy. By combining semantic and keyword-based retrieval methods, the system was able to produce more relevant and reliable responses. Additionally, the inclusion of source citations improved transparency and allows users to seek more information if the one the Chatbot provides is not well enough.

6 Challenges

Several challenges were encountered during the development process. One of the primary issues was the high computational cost of vectorization, which initially resulted in slow response times. This was resolved by moving the vectorization process into its own file to only be executed once.

Additional challenges included finding a Python version that supported the Langchain framework used and finding appropriate packages within the Langchain framework.

Despite these challenges, the team was able to find and optimize a workflow that resulted in performance improvements.

7 Future Work

Future improvements to the Chatbot include the development of an administrative interface that allows for document management, improving the scalability of the Chatbot.

Further optimization can be achieved by integrating more advanced language models, including cloud-based APIs that could improve the computing time.

Enhancing the user interface and deploying the system as a fully functional mobile app are also on the docket for this project.

8 Conclusion

This project demonstrates the successful implementation of a Retrieval-Augmented Generation Chatbot for academic advising in the Computer Science department at Georgia State University. By combining document retrieval techniques with a language model, the system is able to generate accurate and context aware responses.

Through optimizations such as precomputed vectorization and hybrid retrieval, the Chatbot achieved significant improvements in both performance and accuracy. The system provides a strong foundation for future enhancements and highlights the potential of RAG-based systems in academic environments.